Coding Sample: Escaping Inequality in India

Selected Excerpts

Rishav Roy

What follows are excerpts from the 138-page code appendix to my independent paper and writing sample, "Escaping Inequality in India: The Role of English-Medium Instruction."

The next 25 pages illustrate data ingestion and harmonization, complex survey econometrics, IV estimation, and careful diagnostics. Each excerpt is a part of the code chunks listed below:

14-16. Scalable **import, cleaning, and merging** of multi-file census and geospatial data.

- Chunk 4: Import key data files

31-32. **QA and identifier-disambiguation checks** for district-level crosswalk.

- Chunk 6: Match districts: Construct district tracking dfs

50-52. **Testing for systematic missingness** using the Benjamini–Hochberg procedure and parallelized logistic regressions.

- Chunk 8: Participation model: Are NAs randomly distributed

55-57. **Survey-weighted probit estimation**, average marginal effects derivation, and setup for sample selection correction under a complex survey design.

- Chunk 9: Participation model: Estimation and IMR
- Chunk 10: Participation model: Derive AME

80-88. Reusable **record-linkage functions** using cascading fuzzy matches and unmatched/false-match diagnostics to harmonize district identifiers across years and data sources.

- Chunk 16: Match districts: Test joining methods
- Chunk 17: Match districts: Define joining functions

105-107. Reusable **IV specification and estimation pipeline** with clustered inference, plus contiguity-based spatial weights and setup for spatially lagged models.

- Chunk 24: Geospatial: Neighbor list construction
- Chunk 25: 2SLS: Controls and 2SLS formula
- Chunk 26: 2SLS: Baseline 2SLS models

111-112. **Robustness checks and diagnostics** for multicollinearity, spatial autocorrelation.

- Chunk 28: Multicollinearity check
- Chunk 29: Geospatial: Spatial autocorrelation tests

Scalable import, cleaning, and merging of Census/geospatial data

Source: `R/io/read_inputs.R`

```
#' read nss 2007 education
#'
read_nss_2007_education <- function(paths) {
  read_manifest_group(paths, "nss_2007_education")
}

#' read nss 2007 consumption
#'
read_nss_2007_consumption <- function(paths) {
  read_manifest_group(paths, "nss_2007_consumption")
}

#' read nss 2017 education
#'
read_nss_2017_education <- function(paths) {
  read_manifest_group(paths, "nss_2017_education")
}

#' read census 2001 mother tongue
#'
read_census_2001_mother_tongue <- function(paths) {
  read_manifest_group(paths, "census_2001_mother_tongue")
}

#' read district boundaries 2020
#'
read_district_boundaries_2020 <- function(paths) {
  rows <- require_manifest_files(paths, "district_boundaries_2020")
  shp <- rows[tolower(rows$file_type) == "shp", , drop = FALSE]
  if (!nrow(shp)) stop("The district-boundary manifest includes shapefile sidecars but no
   ↪  .shp row.", call. = FALSE)
  need_pkg("sf", "district boundaries")
  sf::st_read(shp$absolute_path[[1]], quiet = TRUE)
}

#' read district change sources
#'
read_district_change_sources <- function(paths) {
  read_manifest_group(paths, "district_changes")
}

#' list ilo figure paths
#'
list_ilo_figure_paths <- function(paths) {
  rows <- require_manifest_files(paths, "ilo_figures")
  stats::setNames(rows$absolute_path, rows$file_id)
}

#' read one manifest row
#'
#' @return Reader output for raw data files, or a validated path for sidecars/assets.
read_by_manifest_row <- function(row) {
```

```
  path <- row$absolute_path[[1]]
  file_type <- tolower(row$file_type[[1]])
  switch(
    file_type,
    sav = read_sav_short(path),
    csv = read_csv_short(path),
    xls = read_excel_short(path),
    xlsx = read_excel_short(path),
    ods = read_ods_short(path),
    shp = {
      need_pkg("sf", "shapefiles")
      sf::st_read(path, quiet = TRUE)
    },
    shx = path,
    dbf = path,
    prj = path,
    png = path,
    stop("No reader implemented for ", file_type, call. = FALSE)
  )
}

#' read all manifest rows for a source
#'
#' @return Named list of reader outputs keyed by manifest file_id.
read_manifest_group <- function(paths, source_id) {
  rows <- require_manifest_files(paths, source_id)
  stats::setNames(lapply(seq_len(nrow(rows)), function(i) read_by_manifest_row(rows[i,
  ↪ ])), rows$file_id)
}
```

QA and identifier-disambiguation checks

Source: `R/districts/build_district_tracker.R`

```
#' build district tracker
#'
build_district_tracker <- function(raw_district_changes) {
  tracker_key <- if ("india_district_tracker" %in% names(raw_district_changes))
  ↪  "india_district_tracker" else "tracker"
  parsed <- list(
    alluvial = parse_alluvial_district_changes(raw_district_changes[["alluvial"]] %||%
    ↪  data.frame()),
    carveouts_renamings =
    ↪  parse_carveouts_renamings(raw_district_changes[["carveouts_renamings"]] %||%
    ↪  data.frame()),
    new_districts_created =
    ↪  parse_new_districts_created(raw_district_changes[["new_districts_created"]] %||%
    ↪  data.frame()),
    name_changes = parse_name_changes(raw_district_changes[["name_changes"]] %||%
    ↪  data.frame()),
    district_splits = parse_district_splits(raw_district_changes[["district_splits"]]
    ↪  %||% data.frame())
  )
  parsed[[tracker_key]] <-
  ↪  parse_india_district_tracker(raw_district_changes[[tracker_key]] %||% data.frame())
```

```
  extras <- setdiff(names(raw_district_changes), c("alluvial", "india_district_tracker",
 ↪  "tracker", "carveouts_renamings", "new_districts_created", "name_changes",
 ↪  "district_splits"))
  for (nm in extras) parsed[[nm]] <-
   ↪  parse_district_change_source(raw_district_changes[[nm]], source_type = nm)
  standardize_tracker_names(standardize_tracker_years(combine_district_tracker_sources
   ↪  (parsed)))
}

#' parse alluvial district changes
#'
parse_alluvial_district_changes <- function(x) {
  parse_district_change_source(x, source_type = "alluvial")
}

#' parse india district tracker
#'
parse_india_district_tracker <- function(x) {
  parse_district_change_source(x, source_type = "india_district_tracker")
}

#' parse carveouts renamings
#'
parse_carveouts_renamings <- function(x) {
  parse_district_change_source(x, source_type = "carveouts_renamings")
}

#' parse new districts created
#'
parse_new_districts_created <- function(x) {
  parse_district_change_source(x, source_type = "new_districts_created")
}

#' parse name changes
#'
parse_name_changes <- function(x) {
  parse_district_change_source(x, source_type = "name_changes")
}

#' parse district splits
#'
parse_district_splits <- function(x) {
  parse_district_change_source(x, source_type = "district_splits")
}

parse_district_change_source <- function(x, source_type) {
  x <- safe_df(x)
  if (!nrow(x)) return(data.frame(source_type = character(), stringsAsFactors = FALSE))
  x$source_type <- source_type
  x$source_state_raw <- first_present_value(x, c("state", "State", "state_raw",
 ↪  "state_from", "state_01", "state_07", "state_08", "state_17", "state_18",
 ↪  "state_20"))
  x$source_district_raw <- first_present_value(x, c("district", "District",
 ↪  "district_raw", "district_from", "district_01", "district_07", "district_08",
 ↪  "district_17", "district_18", "district_20", "district_name"))
```

```
  x$target_state_raw <- first_present_value(x, c("state_to", "new_state", "target_state",
 ↪  "state_20", "state_18", "state_17", "state_08", "state_07", "state"))
  x$target_district_raw <- first_present_value(x, c("district_to", "new_district",
 ↪  "target_district", "district_20", "district_18", "district_17", "district_08",
 ↪  "district_07", "district"))
  x$source_year_raw <- first_present_value(x, c("source_year", "year_from", "from_year",
 ↪  "start_year", "year"))
  x$target_year_raw <- first_present_value(x, c("target_year", "year_to", "to_year",
 ↪  "end_year", "year"))
  x$change_type <- first_present_value(x, c("change_type", "type", "event_type",
 ↪  "status"))
  x
}

first_present_value <- function(df, candidates) {
  hit <- first_col(df, candidates)
  if (is.null(hit)) return(rep(NA_character_, nrow(df)))
  as.character(df[[hit]])
}

#' combine district tracker sources
#'
combine_district_tracker_sources <- function(raw_district_changes) {
  out <- safe_bind_rows(lapply(names(raw_district_changes), function(name) {
    x <- safe_df(raw_district_changes[[name]])
    if (!nrow(x)) return(data.frame())
    x$source_file_id <- if ("source_file_id" %in% names(x)) x$source_file_id else name
    if (!"source_type" %in% names(x)) x$source_type <- name
    x
  }))
  if (nrow(out)) out$.row_in_source <- ave(seq_len(nrow(out)), out$source_file_id, FUN =
   ↪  seq_along)
  out
}

#' standardize tracker years
#'
standardize_tracker_years <- function(tracker) {
  tracker <- safe_df(tracker)
  if (!nrow(tracker)) return(tracker)
  if ("source_year_raw" %in% names(tracker) && !"source_year" %in% names(tracker))
   ↪  tracker$source_year <- suppressWarnings(as.integer(tracker$source_year_raw))
  if ("target_year_raw" %in% names(tracker) && !"target_year" %in% names(tracker))
   ↪  tracker$target_year <- suppressWarnings(as.integer(tracker$target_year_raw))
  tracker
}

#' standardize tracker names
#'
standardize_tracker_names <- function(tracker) {
  tracker <- safe_df(tracker)
  if (!nrow(tracker)) return(tracker)
  if ("source_state_raw" %in% names(tracker)) tracker$source_state_key <-
   ↪  canonicalize_state_name(tracker$source_state_raw)
```

```
  if ("source_district_raw" %in% names(tracker)) tracker$source_district_key <-
   ↪  canon(tracker$source_district_raw)
  if ("target_state_raw" %in% names(tracker)) tracker$target_state_key <-
   ↪  canonicalize_state_name(tracker$target_state_raw)
  if ("target_district_raw" %in% names(tracker)) tracker$target_district_key <-
   ↪  canon(tracker$target_district_raw)
  tracker
}
```

Parallelized missingness diagnostics with BH correction

Source: R/selection/diagnose_missingness.R

```
missingness_variables <- function(selection_data) {
  df <- as.data.frame(selection_data, stringsAsFactors = FALSE)
  all_child_missing_vars <- c("DIST_FROM_NEAREST_PRIMARY_CLASS",
 ↪  "dmean_num_ENROLLMENT_COST", "father_educ")
  enrolled_only_missing_vars <- c("TUTION_FEE", "EXAMINATION_FEE", "OTHER_FEES_PAYMENTS",
 ↪  "BOOKS", "STATIONERY", "UNIFORM", "TRANSPORT")

  # Legacy Chunk 8 distinguished variables used in the probit from
  # enrolled-only expenditure variables.  Keep that distinction so the total
  # row labelled "probit-model" is not inflated by fee variables that are
  # undefined for non-enrolled children.
  probit_vars <- c(
    "enrolled", "ENROLLED", "AGE", "age", "SEX", "HH_SIZE", "RELIGION", "SOCIAL_GROUP",
    "SECTOR", "state_0708", "region_0708", all_child_missing_vars
  )
  probit_vars <- intersect(probit_vars, names(df))
  if (!length(probit_vars)) probit_vars <- setdiff(names(df),
   ↪  intersect(enrolled_only_missing_vars, names(df)))

  list(
    probit_vars = probit_vars,
    miss_vars_all = intersect(all_child_missing_vars, names(df)),
    miss_vars_enrolled = intersect(enrolled_only_missing_vars, names(df)),
    group_vars = intersect(c("SECTOR", "SEX", "RELIGION", "SOCIAL_GROUP", "state_0708"),
     ↪  names(df)),
    cts_vars = intersect(c("AGE", "HH_SIZE"), names(df)),
    regional_vars = intersect(c("state_0708", "region_0708"), names(df))
  )
}

#' diagnose missingness
#'
#' Port the legacy Chunk 8 missingness diagnostics into an opt-in diagnostic
#' object.  The returned list preserves the legacy sequence: variable counts,
#' regional rankings, missingness correlation matrices, BH-adjusted logit screens,
#' and notes for commented case-study / chi-square checks.
diagnose_missingness <- function(selection_data, cfg) {
  df <- as.data.frame(selection_data, stringsAsFactors = FALSE)
  vars <- missingness_variables(df)
  probit_vars <- vars$probit_vars

  counts <- summarize_missingness_by_variable(df[probit_vars])
```

```r
if (length(probit_vars)) {
  total_na <- sum(!stats::complete.cases(df[probit_vars]))
  total_complete <- sum(stats::complete.cases(df[probit_vars]))
} else {
  total_na <- 0L
  total_complete <- nrow(df)
}
counts <- safe_bind_rows(list(
  counts,
  data.frame(missing_var = "Total probit-model with NA", n_missing = total_na,
   ↪  pct_missing = if (nrow(df)) total_na / nrow(df) else NA_real_),
  data.frame(missing_var = "Total probit-model with no NA", n_missing = total_complete,
   ↪  pct_missing = if (nrow(df)) total_complete / nrow(df) else NA_real_)
))

regional <- summarize_missingness_regions(df, probit_vars, vars)
corr_all <- missingness_correlation_matrix(df, vars$miss_vars_all, vars$group_vars,
↪  vars$cts_vars)
enrolled_rows <- enrolled_schooling_rows(df)
corr_enrolled <- missingness_correlation_matrix(df[enrolled_rows, , drop = FALSE],
↪  vars$miss_vars_enrolled, vars$group_vars, vars$cts_vars)

covars <- intersect(c("SECTOR", "SEX", "AGE", "HH_SIZE", "RELIGION", "SOCIAL_GROUP",
↪  "state_0708"), names(df))
miss_all <- intersect(vars$miss_vars_all, names(df))
miss_enrolled <- intersect(vars$miss_vars_enrolled, names(df))
logit_all <- if (length(miss_all) && length(covars)) run_missingness_logits(df,
↪  miss_all, covars) else data.frame()
logit_enrolled <- if (length(miss_enrolled) && length(covars) && any(enrolled_rows))
↪  run_missingness_logits(df[enrolled_rows, , drop = FALSE], miss_enrolled, covars) else
↪  data.frame()
logit_summary <- summarize_missingness_logits(safe_bind_rows(list(logit_all,
↪  logit_enrolled)))
case_study <- summarize_missingness_case_study(df, vars)
chi_square <- summarize_missingness_chisq(df, probit_vars)

notes <- data.frame(
  diagnostic = c(
    "rajasthan_southern_case_study",
    "chi_square_any_na_by_state",
    "chi_square_any_na_by_region",
    "parallel_missingness_logit"
  ),
  legacy_status = c(
    "commented diagnostic View() code preserved as documented note",
    "commented diagnostic test not run by default",
    "commented diagnostic test not run by default",
    "ported using lapply for reproducible targets execution; legacy mclapply/parLapply
     ↪  choice documented"
  ),
  stringsAsFactors = FALSE
)

out <- list(
  missing_counts = counts,
```

```
    regional_cost = regional$cost,
    regional_distance = regional$distance,
    regional_father_education = regional$father_education,
    corr_all = corr_all,
    corr_enrolled = corr_enrolled,
    logit_all = logit_all,
    logit_enrolled = logit_enrolled,
    logit_summary = logit_summary,
    case_study = case_study,
    chi_square = chi_square,
    notes = notes
  )
  class(out) <- c("emi_missingness_diagnostics", class(out))
  out
}

enrolled_schooling_rows <- function(df) {
  if (!"enrolled" %in% names(df)) return(rep(TRUE, nrow(df)))
  value <- tolower(as.character(df$enrolled))
  value %in% c("yes", "1", "true", "enrolled")
}

summarize_missingness_by_variable <- function(df) {
  df <- as.data.frame(df, stringsAsFactors = FALSE)
  if (!length(names(df))) {
    return(data.frame(missing_var = character(), n_missing = integer(), pct_missing =
     ↪  numeric(), stringsAsFactors = FALSE))
  }
  data.frame(
    missing_var = names(df),
    n_missing = vapply(df, function(x) sum(is.na(x)), integer(1)),
    pct_missing = if (nrow(df)) vapply(df, function(x) mean(is.na(x)), numeric(1)) else
     ↪  NA_real_,
    stringsAsFactors = FALSE
  )
}


summarize_missingness_case_study <- function(df, vars) {
  df <- as.data.frame(df, stringsAsFactors = FALSE)
  if (!nrow(df)) return(data.frame())
  state_col <- intersect(c("state_0708", "state"), names(df))
  region_col <- intersect(c("region_0708", "region"), names(df))
  if (!length(state_col)) return(data.frame(note = "No state column available for
   ↪  Rajasthan/Southern case study.", stringsAsFactors = FALSE))

  keep <- grepl("rajasthan", as.character(df[[state_col[[1]]]]), ignore.case = TRUE)
  case_scope <- "Rajasthan"
  if (length(region_col)) {
    region_keep <- grepl("southern", as.character(df[[region_col[[1]]]]), ignore.case =
↪  TRUE)
    if (any(keep & region_keep, na.rm = TRUE)) {
      keep <- keep & region_keep
      case_scope <- "Rajasthan / Southern"
    }
```

```
  }
  vars_to_check <- intersect(c(vars$miss_vars_all, vars$miss_vars_enrolled), names(df))
  if (!length(vars_to_check) || !any(keep, na.rm = TRUE)) {
    return(data.frame(case_scope = case_scope, n_rows = sum(keep, na.rm = TRUE), note =
     ↪  "No matching rows or missingness variables available.", stringsAsFactors =
     ↪  FALSE))
  }
  case_df <- df[keep, vars_to_check, drop = FALSE]
  any_missing <- !stats::complete.cases(case_df)
  data.frame(
    case_scope = case_scope,
    n_rows = nrow(case_df),
    n_rows_with_any_missing = sum(any_missing),
    n_rows_with_partial_missing = sum(any_missing & rowSums(is.na(case_df)) <
     ↪  ncol(case_df)),
    n_missing_cells = sum(is.na(case_df)),
    n_observed_cells_in_rows_with_missing = sum(!is.na(case_df[any_missing, , drop =
     ↪  FALSE])),
    interpretation = "Current analog of the legacy Rajasthan/Southern View() check: rows
     ↪  with partial cost-variable missingness preserve the legacy concern that a child
     ↪  was not necessarily excluded wholesale when one cost field was missing.",
    stringsAsFactors = FALSE
  )
}

summarize_missingness_chisq <- function(df, probit_vars) {
  df <- as.data.frame(df, stringsAsFactors = FALSE)
  if (!nrow(df) || !length(probit_vars)) return(data.frame())
  probit_vars <- intersect(probit_vars, names(df))
  if (!length(probit_vars)) return(data.frame())
  any_na <- !stats::complete.cases(df[probit_vars])
  safe_test <- function(group_col, label) {
    if (!group_col %in% names(df)) {
      return(data.frame(test = label, status = "not_available", reason = paste("Missing
       ↪  column", group_col), stringsAsFactors = FALSE))
    }
    group <- as.character(df[[group_col]])
    keep <- !is.na(group) & nzchar(group)
    if (length(unique(group[keep])) < 2L || length(unique(any_na[keep])) < 2L) {
      return(data.frame(test = label, status = "not_estimated", reason = "Insufficient
       ↪  variation for chi-square test.", stringsAsFactors = FALSE))
    }
    tab <- table(any_na = any_na[keep], group = group[keep])
    fit <- suppressWarnings(stats::chisq.test(tab))
    data.frame(
      test = label,
      status = "estimated",
      statistic = unname(fit$statistic),
      parameter = unname(fit$parameter),
      p.value = unname(fit$p.value),
      n = sum(tab),
      method = fit$method,
      stringsAsFactors = FALSE
    )
  }
```

```
  safe_bind_rows(list(
    safe_test("state_0708", "any_probit_model_na_by_state"),
    safe_test("region_0708", "any_probit_model_na_by_region")
  ))
}

summarize_missingness_regions <- function(df, probit_vars, vars, top_n = 20L) {
  empty <- data.frame()
  if (!"state_0708" %in% names(df)) {
    return(list(cost = empty, distance = empty, father_education = empty))
  }
  if (!length(probit_vars)) probit_vars <- names(df)
  temp <- df
  region_cols <- intersect(c("state_0708", "region_0708"), names(temp))
  # Legacy Chunk 8 inspected both state and region.  Some cleaned selection
  # inputs no longer expose region_0708, so keep the diagnostic alive at the
  # state level instead of writing empty CSVs.
  if (!"region_0708" %in% region_cols) {
    temp$region_0708 <- "all_regions_available_in_state_only_input"
    region_cols <- c("state_0708", "region_0708")
  }

  temp$any_na_row <- !stats::complete.cases(temp[intersect(probit_vars, names(temp))])
  for (nm in c("dmean_num_ENROLLMENT_COST", "DIST_FROM_NEAREST_PRIMARY_CLASS",
   ↪  "father_educ")) {
    temp[[paste0("miss_", nm)]] <- if (nm %in% names(temp)) as.integer(is.na(temp[[nm]]))
  ↪  else NA_integer_
  }
  temp$is_urban <- if ("SECTOR" %in% names(temp)) as.integer(temp$SECTOR == "Urban") else
  ↪  NA_integer_
  temp$is_female <- if ("SEX" %in% names(temp)) as.integer(temp$SEX == "Female") else
  ↪  NA_integer_
  temp$is_hindu <- if ("RELIGION" %in% names(temp)) as.integer(temp$RELIGION == "Hindu")
  ↪  else NA_integer_
  temp$is_muslim <- if ("RELIGION" %in% names(temp)) as.integer(temp$RELIGION ==
  ↪  "Muslim") else NA_integer_
  temp$is_st_sc_obc <- if ("SOCIAL_GROUP" %in% names(temp)) as.integer(temp$SOCIAL_GROUP
  ↪  %in% c("Scheduled Tribe", "Scheduled Caste", "Other Backward Class")) else
  ↪  NA_integer_

  measure_cols <- c("any_na_row", "miss_dmean_num_ENROLLMENT_COST",
  ↪  "miss_DIST_FROM_NEAREST_PRIMARY_CLASS", "miss_father_educ", "is_urban", "is_female",
  ↪  "is_hindu", "is_muslim", "is_st_sc_obc")
  grouped <- stats::aggregate(
    temp[measure_cols],
    temp[region_cols],
    function(x) mean(as.numeric(x), na.rm = TRUE)
  )
  n <- stats::aggregate(temp$any_na_row, temp[region_cols], length)
  names(n)[names(n) == "x"] <- "n"
  out <- merge(n, grouped, by = region_cols, all = TRUE)
  names(out) <- sub("^any_na_row$", "pct_any_na", names(out))
  out$region_diagnostic_level <- if ("region_0708" %in% names(df)) "state_region" else
  ↪  "state_only_fallback"
  pct_cols <- setdiff(names(out), c(region_cols, "n", "region_diagnostic_level"))
```

```r
  out[pct_cols] <- lapply(out[pct_cols], function(x) round(100 * x, 2))
  rank_one <- function(col) {
    if (!col %in% names(out)) return(empty)
    out[order(-out[[col]]), , drop = FALSE][seq_len(min(top_n, nrow(out))), , drop =
↪   FALSE]
  }
  list(
    cost = rank_one("miss_dmean_num_ENROLLMENT_COST"),
    distance = rank_one("miss_DIST_FROM_NEAREST_PRIMARY_CLASS"),
    father_education = rank_one("miss_father_educ")
  )
}

missingness_correlation_matrix <- function(df, miss_vars, group_vars, cts_vars) {
  df <- as.data.frame(df, stringsAsFactors = FALSE)
  miss_vars <- intersect(miss_vars, names(df))
  if (!length(miss_vars) || !nrow(df)) return(matrix(numeric(), nrow = 0L, ncol = 0L))
  miss_df <- as.data.frame(lapply(df[miss_vars], function(x) as.integer(is.na(x))),
↪   check.names = FALSE)
  names(miss_df) <- paste0("miss_", names(miss_df))
  cts <- intersect(cts_vars, names(df))
  cts_df <- if (length(cts)) as.data.frame(lapply(df[cts], numeric_like), check.names =
↪   FALSE) else data.frame()
  grp <- intersect(group_vars, names(df))
  grp_df <- if (length(grp)) {
    grp_data <- as.data.frame(lapply(df[grp], function(x) {
      x <- as.character(x)
      x[is.na(x) | !nzchar(x)] <- "missing"
      factor(x)
    }), check.names = FALSE)
    # Legacy Chunk 8 used model.matrix-style expansions of sector, sex,
    # religion, social group, state, and related grouping variables before
    # computing missingness correlations.  Small diagnostic subsets, especially
    # enrolled-only slices used in tests or regional filters, may contain only a
    # single observed level for one or more grouping variables.  model.matrix()
    # cannot apply contrasts to one-level factors, so drop constant grouping
    # variables while preserving all grouping variables with real variation.
    varying <- vapply(grp_data, function(x) length(unique(as.character(x))) > 1L,
↪   logical(1))
    grp_data <- grp_data[varying]
    if (length(grp_data)) {
      stats::model.matrix(~ . - 1, data = grp_data)
    } else {
      matrix(numeric(), nrow = nrow(df), ncol = 0L)
    }
  } else matrix(numeric(), nrow = nrow(df), ncol = 0L)
  safe_pairwise_cor(cbind(miss_df, cts_df, grp_df))
}

#' check missing logit parallel
#'
#' Legacy Chunk 8 used mclapply/parLapply after defining the same one-logit-per-
#' missing-variable problem.  Targets already parallelizes at the target level,
#' so this implementation keeps deterministic in-process lapply while preserving
#' the model specification and BH adjustment.
```

```
check_missing_logit_parallel <- function(df, miss_vars, covars, method_p = "BH") {
  df <- as.data.frame(df, stringsAsFactors = FALSE)
  miss_vars <- intersect(miss_vars, names(df))
  covars <- intersect(covars, names(df))
  if (!length(miss_vars) || !length(covars)) return(data.frame())
  rhs <- paste(covars, collapse = " + ")
  fit_one <- function(m) {
    y <- is.na(df[[m]])
    if (length(unique(y)) < 2L) return(data.frame())
    f <- stats::as.formula(paste0("is.na(`", m, "`) ~ ", rhs))
    tryCatch({
      fit <- stats::glm(f, data = df, family = stats::binomial)
      pseudoR2 <- 1 - fit$deviance / fit$null.deviance
      out <- broom::tidy(fit)
      out$missing_var <- m
      out$pseudoR2 <- pseudoR2
      out$nobs <- stats::nobs(fit)
      out
    }, error = function(e) {
      data.frame(term = NA_character_, estimate = NA_real_, std.error = NA_real_,
       ↪  statistic = NA_real_, p.value = NA_real_, missing_var = m, pseudoR2 = NA_real_,
       ↪  nobs = length(y), status = "failed", reason = conditionMessage(e),
       ↪  stringsAsFactors = FALSE)
    })
  }
  out <- safe_bind_rows(lapply(miss_vars, fit_one))
  adjust_missingness_pvalues_bh(out, method_p = method_p)
}

run_missingness_logits <- function(df, miss_vars, covars) {
  check_missing_logit_parallel(df, miss_vars, covars)
}

adjust_missingness_pvalues_bh <- function(df, method_p = "BH") {
  df <- as.data.frame(df, stringsAsFactors = FALSE)
  if (!nrow(df) || !"p.value" %in% names(df)) return(df)
  df$p_adj <- NA_real_
  idx <- !is.na(df$p.value) & df$term != "(Intercept)"
  df$p_adj[idx] <- stats::p.adjust(df$p.value[idx], method = method_p)
  df
}

summarize_missingness_logits <- function(df) {
  df <- as.data.frame(df, stringsAsFactors = FALSE)
  if (!nrow(df) || !all(c("missing_var", "p_adj", "pseudoR2") %in% names(df))) {
    return(data.frame(missing_var = character(), n_sig = integer(), pseudoR2 = numeric(),
     ↪  stringsAsFactors = FALSE))
  }
  split_df <- split(df, df$missing_var)
  out <- safe_bind_rows(lapply(split_df, function(x) {
    pmax <- suppressWarnings(max(x$pseudoR2, na.rm = TRUE))
    if (!is.finite(pmax)) pmax <- NA_real_
    data.frame(
      missing_var = x$missing_var[[1]],
      n_sig = sum(x$p_adj < 0.05 & x$term != "(Intercept)", na.rm = TRUE),
```

```
      pseudoR2 = pmax,
      stringsAsFactors = FALSE
    )
  }))
  out[order(-out$pseudoR2), , drop = FALSE]
}


missingness_correlation_pairs <- function(mat, top_n = 50L) {
  mat <- as.matrix(mat)
  if (!length(mat) || nrow(mat) < 2L || ncol(mat) < 2L) {
    return(data.frame(var1 = character(), var2 = character(), correlation = numeric(),
    ↪  abs_correlation = numeric(), stringsAsFactors = FALSE))
  }
  mat[lower.tri(mat, diag = TRUE)] <- NA_real_
  idx <- which(is.finite(mat), arr.ind = TRUE)
  if (!nrow(idx)) {
    return(data.frame(var1 = character(), var2 = character(), correlation = numeric(),
    ↪  abs_correlation = numeric(), stringsAsFactors = FALSE))
  }
  out <- data.frame(
    var1 = rownames(mat)[idx[, "row"]],
    var2 = colnames(mat)[idx[, "col"]],
    correlation = mat[idx],
    stringsAsFactors = FALSE
  )
  out$abs_correlation <- abs(out$correlation)
  out <- out[order(-out$abs_correlation), , drop = FALSE]
  utils::head(out, top_n)
}

save_missingness_correlation_heatmap <- function(mat, path, max_vars = 35L, title =
↪  "Missingness correlation matrix") {
  mat <- as.matrix(mat)
  dir.create(dirname(path), recursive = TRUE, showWarnings = FALSE)
  if (!length(mat) || nrow(mat) < 2L || ncol(mat) < 2L) {
    grDevices::png(path, width = 900, height = 500, res = 120)
    graphics::plot.new()
    graphics::text(0.5, 0.5, "No correlation matrix available")
    grDevices::dev.off()
    return(normalizePath(path, mustWork = FALSE))
  }
  score <- apply(abs(mat), 1L, function(x) max(x[is.finite(x) & x < 1], na.rm = TRUE))
  score[!is.finite(score)] <- 0
  keep <- names(sort(score, decreasing = TRUE))[seq_len(min(max_vars, length(score)))]
  mat <- mat[keep, keep, drop = FALSE]
  grDevices::png(path, width = 1400, height = 1200, res = 140)
  old <- graphics::par(no.readonly = TRUE)
  on.exit({ graphics::par(old); grDevices::dev.off() }, add = TRUE)
  graphics::par(mar = c(11, 11, 4, 2))
  graphics::image(
    x = seq_len(ncol(mat)),
    y = seq_len(nrow(mat)),
    z = t(mat[nrow(mat):1, , drop = FALSE]),
    axes = FALSE,
```

```
    xlab = "",
    ylab = "",
    main = title,
    zlim = c(-1, 1)
  )
  graphics::axis(1, at = seq_len(ncol(mat)), labels = colnames(mat), las = 2, cex.axis =
 ↪  0.55)
  graphics::axis(2, at = seq_len(nrow(mat)), labels = rev(rownames(mat)), las = 2,
 ↪  cex.axis = 0.55)
  graphics::box()
  normalizePath(path, mustWork = FALSE)
}


save_missingness_logit_plot <- function(summary_df, path, title = "Structured missingness
 ↪  logit screen", top_n = 25L) {
  dir.create(dirname(path), recursive = TRUE, showWarnings = FALSE)
  df <- as.data.frame(summary_df, stringsAsFactors = FALSE)
  grDevices::png(path, width = 1100, height = 850, res = 130)
  old <- graphics::par(no.readonly = TRUE)
  on.exit({ graphics::par(old); grDevices::dev.off() }, add = TRUE)
  if (!nrow(df) || !all(c("missing_var", "pseudoR2") %in% names(df))) {
    graphics::plot.new()
    graphics::text(0.5, 0.5, "No missingness-logit summary available")
    return(normalizePath(path, mustWork = FALSE))
  }
  df <- df[is.finite(df$pseudoR2), , drop = FALSE]
  if (!nrow(df)) {
    graphics::plot.new()
    graphics::text(0.5, 0.5, "No finite pseudo-R² values available")
    return(normalizePath(path, mustWork = FALSE))
  }
  df <- df[order(df$pseudoR2, decreasing = TRUE), , drop = FALSE]
  df <- utils::head(df, top_n)
  graphics::par(mar = c(5, 13, 4, 2))
  graphics::barplot(
    rev(df$pseudoR2),
    names.arg = rev(df$missing_var),
    horiz = TRUE,
    las = 1,
    xlab = "Pseudo-R² (predictability of missingness)",
    main = title
  )
  normalizePath(path, mustWork = FALSE)
}

save_missingness_diagnostics <- function(diagnostics, dir =
 ↪  "outputs/diagnostics/extended/missingness") {
  dir.create(dir, recursive = TRUE, showWarnings = FALSE)
  if (!inherits(diagnostics, "emi_missingness_diagnostics")) diagnostics <-
   ↪  list(missing_counts = as.data.frame(diagnostics))
  paths <- c(
    missing_counts = write_diagnostic_csv(diagnostics$missing_counts %||% data.frame(),
     ↪  file.path(dir, "missingness_counts.csv")),
```

```
  regional_cost = write_diagnostic_csv(diagnostics$regional_cost %||% data.frame(),
   ↪  file.path(dir, "regional_missingness_cost.csv")),
  regional_distance = write_diagnostic_csv(diagnostics$regional_distance %||%
   ↪  data.frame(), file.path(dir, "regional_missingness_distance.csv")),
  regional_father = write_diagnostic_csv(diagnostics$regional_father_education %||%
   ↪  data.frame(), file.path(dir, "regional_missingness_father_education.csv")),
  logit_all = write_diagnostic_csv(diagnostics$logit_all %||% data.frame(),
   ↪  file.path(dir, "missingness_logits_all.csv")),
  logit_enrolled = write_diagnostic_csv(diagnostics$logit_enrolled %||% data.frame(),
   ↪  file.path(dir, "missingness_logits_enrolled.csv")),
  logit_summary = write_diagnostic_csv(diagnostics$logit_summary %||% data.frame(),
   ↪  file.path(dir, "missingness_logit_summary.csv")),
  case_study = write_diagnostic_csv(diagnostics$case_study %||% data.frame(),
   ↪  file.path(dir, "missingness_rajasthan_southern_case_study.csv")),
  chi_square = write_diagnostic_csv(diagnostics$chi_square %||% data.frame(),
   ↪  file.path(dir, "missingness_chi_square_tests.csv")),
  notes = write_diagnostic_csv(diagnostics$notes %||% data.frame(), file.path(dir,
   ↪  "missingness_legacy_notes.csv"))
)
if (length(diagnostics$corr_all)) {
  paths <- c(
    paths,
    corr_all = write_diagnostic_matrix(diagnostics$corr_all, file.path(dir,
     ↪  "missingness_correlation_all.csv")),
    corr_all_pairs =
     ↪  write_diagnostic_csv(missingness_correlation_pairs(diagnostics$corr_all),
     ↪  file.path(dir, "missingness_correlation_all_top_pairs.csv")),
    corr_all_heatmap = save_missingness_correlation_heatmap(diagnostics$corr_all,
     ↪  file.path(dir, "missingness_correlation_all.png"), title = "Missingness
     ↪  correlations: all observations")
  )
}
if (length(diagnostics$corr_enrolled)) {
  paths <- c(
    paths,
    corr_enrolled = write_diagnostic_matrix(diagnostics$corr_enrolled, file.path(dir,
     ↪  "missingness_correlation_enrolled.csv")),
    corr_enrolled_pairs =
     ↪  write_diagnostic_csv(missingness_correlation_pairs(diagnostics$corr_enrolled),
     ↪  file.path(dir, "missingness_correlation_enrolled_top_pairs.csv")),
    corr_enrolled_heatmap =
     ↪  save_missingness_correlation_heatmap(diagnostics$corr_enrolled, file.path(dir,
     ↪  "missingness_correlation_enrolled.png"), title = "Missingness correlations:
     ↪  enrolled observations")
  )
}
if (nrow(diagnostics$logit_summary %||% data.frame())) {
  paths <- c(
    paths,
    logit_pseudo_r2_plot = save_missingness_logit_plot(
      diagnostics$logit_summary,
      file.path(dir, "missingness_logit_pseudo_r2.png"),
      title = "How structured is missingness?"
    )
  )
```

```
  }
  output_manifest(paths)
}
```

Survey-weighted probit and IMR setup

Source: `R/selection/estimate_selection_probit.R`

```
selection_probit_variables <- function(selection_data) {
  controls <- c("AGE", "age", "SEX", "HH_SIZE", "RELIGION", "SOCIAL_GROUP", "SECTOR")
  exclusion_all_kids <- c("DIST_FROM_NEAREST_PRIMARY_CLASS", "father_educ")
  exclusion_district <- c(
    "dmean_num_IS_EDU_FREE", "dmean_num_TUTION_FEE_WAIVED",
    "dmean_num_RECD_SCHOLARSHIP_STIPEND", "dmean_num_RECD_TXT_BOOKS",
    "dmean_num_RECD_STATIONERY", "dmean_num_MID_DAY_MEAL_ETC_RECD",
    "dmean_num_ENROLLMENT_COST"
  )
  intersect(c(controls, exclusion_all_kids, exclusion_district), names(selection_data))
}

#' estimate selection probit
#'
estimate_selection_probit <- function(selection_data, cfg) {
  if (!"enrolled" %in% names(selection_data) || all(is.na(selection_data$enrolled))) {
    return(list(status = "out_of_active_pipeline", reason = "No enrolled variable."))
  }
  covars <- selection_probit_variables(selection_data)
  if (!length(covars)) {
    return(list(status = "out_of_active_pipeline", reason = "No probit covariates."))
  }
  f_probit <- stats::reformulate(covars, response = "enrolled")
  if (identical(cfg$mode, "final") && requireNamespace("survey", quietly = TRUE)) {
    design <- build_survey_design_selection(selection_data)
    if (!is.null(design)) {
      out <- fit_selection_probit(design, f_probit)
      attr(out, "selection_probit_formula") <- f_probit
      return(out)
    }
  }
  out <- stats::glm(f_probit, data = selection_data, family = stats::binomial(link =
↪   "probit"))
  attr(out, "selection_probit_formula") <- f_probit
  out
}

#' build survey design selection
#'
build_survey_design_selection <- function(selection_df) {
  psu <- first_col(selection_df, c("FSU_SL_NO", "fsu", "PSU", "psu"))
  weight <- first_col(selection_df, c("weight", "WEIGHT", "Multiplier", "multiplier"))
  strata_cols <- intersect(c("STATE", "STRATUM", "SUB_STRATUM_NO"), names(selection_df))
  if (is.null(psu) || is.null(weight) || !length(strata_cols)) return(NULL)
  options(survey.lonely.psu = "average")
  selection_df$.survey_strata <- interaction(selection_df[strata_cols], drop = TRUE)
  survey::svydesign(
    ids = stats::as.formula(paste0("~", psu)),
```

```
    strata = ~.survey_strata,
    weights = stats::as.formula(paste0("~", weight)),
    data = selection_df,
    nest = TRUE
  )
}

#' fit selection probit
#'
fit_selection_probit <- function(selection_design, f_probit) {
  survey::svyglm(f_probit, design = selection_design, family = stats::quasibinomial(link
 ↪  = "probit"))
}

#' compute inverse mills ratio
#'
compute_inverse_mills_ratio <- function(model, selection_df, f_probit = NULL) {
  if (is.null(f_probit)) f_probit <- attr(model, "selection_probit_formula") %||%
   ↪  stats::formula(model)
  needed <- all.vars(f_probit)
  needed <- intersect(needed, names(selection_df))
  comp_cases <- stats::complete.cases(selection_df[, needed, drop = FALSE])
  lp <- rep(NA_real_, nrow(selection_df))
  lp[comp_cases] <- stats::predict(model, newdata = selection_df[comp_cases, , drop =
 ↪  FALSE], type = "link")
  phi_lp <- stats::dnorm(lp)
  Phi_lp <- pmin(pmax(stats::pnorm(lp), .Machine$double.eps), 1 - .Machine$double.eps)
  enrolled <- as.character(selection_df$enrolled)
  selection_df$IMR <- ifelse(enrolled == "Yes", phi_lp / Phi_lp, ifelse(enrolled == "No",
 ↪  phi_lp / (1 - Phi_lp), NA_real_))
  selection_df
}

#' tidy selection model
#'
tidy_selection_model <- function(model) {
  broom::tidy(model)
}
```

Average marginal effects via automatic differentiation

Source: `R/selection/compute_average_marginal_effects.R`

```
# Use automatic differentiation for SE delta-method gradients, as in the legacy Rmd.

#' compute average marginal effects
#'
compute_average_marginal_effects <- function(selection_model, selection_data, cfg =
 ↪  list()) {
  if (!inherits(selection_model, "glm")) {
    return(ame_out_of_pipeline(
      selection_model$status %||% "out_of_active_pipeline",
      selection_model$reason %||% "Selection model not estimated in smoke mode"
    ))
  }
```

```r
  # `selection_model` is often a survey::svyglm object loaded from the targets
  # store. Survey lonely-PSU handling is an R option, not a durable model
  # attribute, so it may not be set in the downstream AME target even though it
  # was set when the model was estimated. Set it explicitly around all AME
  # computations to make reruns deterministic and warning/error-free.
  old_lonely_psu <- getOption("survey.lonely.psu")
  old_domain_lonely <- getOption("survey.adjust.domain.lonely")
  options(survey.lonely.psu = "average", survey.adjust.domain.lonely = TRUE)
  on.exit({
    if (is.null(old_lonely_psu)) {
      options(survey.lonely.psu = NULL)
    } else {
      options(survey.lonely.psu = old_lonely_psu)
    }
    if (is.null(old_domain_lonely)) {
      options(survey.adjust.domain.lonely = NULL)
    } else {
      options(survey.adjust.domain.lonely = old_domain_lonely)
    }
  }, add = TRUE)

#' run expression while muffling survey lonely-PSU warnings
#'
#' survey emits lonely-PSU warnings even when survey.lonely.psu is set to
#' average and the adjustment is intentional. Muffle only those handled
#' warnings inside the AME target so strict final builds remain warning-clean.
muffle_survey_lonely_psu_warnings <- function(expr) {
  withCallingHandlers(
    expr,
    warning = function(w) {
      msg <- conditionMessage(w)
      lonely_psu_warning <- grepl(
        "has only one PSU at stage",
        msg,
        fixed = TRUE
      )
      if (lonely_psu_warning) invokeRestart("muffleWarning")
    }
  )
}


  if (!isTRUE(cfg$run_full_ame)) {
    return(format_ame_results(data.frame(
      term = names(stats::coef(selection_model)),
      estimate = unname(stats::coef(selection_model)),
      std.error = NA_real_,
      statistic = NA_real_,
      p.value = NA_real_,
      s.value = NA_real_,
      conf.low = NA_real_,
      conf.high = NA_real_,
      method = "coefficient_fallback",
      status = "estimated",
      reason = "Draft config uses coefficient fallback instead of full AME computation"
```

```
    )))
  }

  if (!requireNamespace("marginaleffects", quietly = TRUE)) {
    return(ame_out_of_pipeline(
      "out_of_active_pipeline",
      "Package marginaleffects is required for full AME computation"
    ))
  }

  out <- muffle_survey_lonely_psu_warnings(
    tryCatch(
      compute_ames_autodiff(selection_model, selection_data),
      error = function(e) {
        fallback <- compute_ames_probit_analytic(selection_model, selection_data)
        fallback$method <- "delta_method_analytic_probit"
        fallback$reason <- NA_character_
        fallback
      }
    )
  )
  format_ame_results(out)
}

ame_out_of_pipeline <- function(status, reason) {
  data.frame(
    term = NA_character_,
    estimate = NA_real_,
    std.error = NA_real_,
    statistic = NA_real_,
    p.value = NA_real_,
    s.value = NA_real_,
    conf.low = NA_real_,
    conf.high = NA_real_,
    method = "not_run",
    status = status,
    reason = reason
  )
}

#' extract model-frame data and AME weights
#'
#' marginaleffects warns, correctly, when weighted/survey models are summarized
#' without an explicit `wts` argument. Build a newdata frame from the exact
#' model estimation sample and attach a synthetic weight column whenever the
#' fitted model exposes usable weights.
ame_model_weight <- function(model_data, model_weights = NULL) {
  weight_cols <- intersect(c("weight", "WEIGHT", "Multiplier", "multiplier",
 ↪  "(weights)"), names(model_data))
  if (length(weight_cols)) {
    return(suppressWarnings(as.numeric(model_data[[weight_cols[[1]]]])))
  }

  if (!is.null(model_weights) && length(model_weights) == nrow(model_data)) {
    return(suppressWarnings(as.numeric(model_weights)))
```

```r
  }

  NULL
}

#' build marginaleffects newdata and weights
#'
#' @return List with `data`, `wts`, and `has_explicit_weights`.
ame_model_data_and_weights <- function(model) {
  model_data <- as.data.frame(stats::model.frame(model))
  model_weights <- tryCatch(stats::weights(model), error = function(e) NULL)
  weight <- ame_model_weight(model_data, model_weights)

  if (!is.null(weight) && length(weight) == nrow(model_data) && any(is.finite(weight) &
   ↪  weight != 1)) {
    model_data$.ame_weight <- weight
    return(list(data = model_data, wts = ".ame_weight", has_explicit_weights = TRUE))
  }

  list(data = model_data, wts = FALSE, has_explicit_weights = FALSE)
}

#' run marginaleffects while muffling only a redundant explicit-weight warning
#'
run_avg_slopes <- function(model, model_data, wts) {
  withCallingHandlers(
    marginaleffects::avg_slopes(
      model,
      newdata = model_data,
      wts = wts,
      vcov = TRUE,
      type = "response"
    ),
    warning = function(w) {
      msg <- conditionMessage(w)
      explicit_weight_warning <- grepl(
        "normally good practice to specify weights using the `wts` argument",
        msg,
        fixed = TRUE
      )
      if (explicit_weight_warning && !identical(wts, FALSE)) {
        invokeRestart("muffleWarning")
      }
    }
  )
}

#' compute ames autodiff
#'
compute_ames_autodiff <- function(model, newdata) {
  # Use the exact estimation sample retained by glm/svyglm. Passing the full
  # pre-model selection_data can have extra rows omitted during model fitting,
  # which makes marginaleffects recycle weights/covariates silently.
  amed <- ame_model_data_and_weights(model)
  run_avg_slopes(model, amed$data, amed$wts)
```

```
}

#' compute ames fast draft
#'
compute_ames_fast_draft <- function(model, newdata, n = 200) {
  amed <- ame_model_data_and_weights(model)
  run_avg_slopes(
    model,
    dplyr::slice_sample(amed$data, n = min(n, nrow(amed$data))),
    amed$wts
  )
}

#' compute analytic probit AMEs
#'
#' @return Data frame of observed-data average marginal effects.
compute_ames_probit_analytic <- function(model, newdata) {
  coefs <- stats::coef(model)
  coef_table <- as.data.frame(summary(model)$coefficients)
  terms <- setdiff(names(coefs), "(Intercept)")
  if (!length(terms)) return(ame_out_of_pipeline("out_of_active_pipeline", "Selection
   ↪  model has no non-intercept terms."))

  newdata <- stats::model.frame(model)
  weights <- if ("weight" %in% names(newdata)) num(newdata$weight) else rep(1,
 ↪  nrow(newdata))
  eta <- stats::predict(model, type = "link")
  mean_phi <- wmean(stats::dnorm(eta), weights)
  factor_levels <- model$xlevels %||% list()

  rows <- lapply(terms, function(term) {
    factor_var <- names(factor_levels)[vapply(names(factor_levels), function(v)
 ↪  startsWith(term, v), logical(1))]
    estimate <- NA_real_
    if (length(factor_var)) {
      var <- factor_var[[which.max(nchar(factor_var))]]
      level <- sub(paste0("^", var), "", term)
      if (level %in% factor_levels[[var]]) {
        hi <- newdata
        lo <- newdata
        hi[[var]] <- factor(level, levels = factor_levels[[var]])
        lo[[var]] <- factor(factor_levels[[var]][[1]], levels = factor_levels[[var]])
        estimate <- wmean(stats::predict(model, newdata = hi, type = "response") -
 ↪  stats::predict(model, newdata = lo, type = "response"), weights)
      }
    } else {
      estimate <- unname(coefs[[term]]) * mean_phi
    }
    p_col <- intersect(c("Pr(>|z|)", "Pr(>|t|)"), names(coef_table))
    p <- if (term %in% rownames(coef_table) && length(p_col)) coef_table[term,
 ↪  p_col[[1]]] else NA_real_
    se_col <- intersect(c("Std. Error", "std.error"), names(coef_table))
    se_beta <- if (term %in% rownames(coef_table) && length(se_col)) coef_table[term,
 ↪  se_col[[1]]] else NA_real_
    se <- if (is.finite(se_beta)) abs(se_beta * mean_phi) else NA_real_
```

```
    z <- if (is.finite(se) && se > 0) estimate / se else NA_real_
    p_out <- if (is.finite(z)) 2 * stats::pnorm(abs(z), lower.tail = FALSE) else p
    data.frame(
      term = term,
      estimate = estimate,
      std.error = se,
      statistic = z,
      p.value = p_out,
      s.value = if (is.finite(p_out) && p_out > 0) -log2(p_out) else NA_real_,
      conf.low = if (is.finite(se)) estimate - 1.96 * se else NA_real_,
      conf.high = if (is.finite(se)) estimate + 1.96 * se else NA_real_,
      method = "delta_method_analytic_probit",
      status = "estimated",
      reason = NA_character_,
      stringsAsFactors = FALSE
    )
  })
  do.call(rbind, rows)
}


is_marginaleffects_object <- function(x) {
  inherits(x, c("marginaleffects", "comparisons", "avg_comparisons", "slopes",
   ↪  "avg_slopes", "predictions"))
}

copy_first_ame_column <- function(out, target, candidates) {
  if (!target %in% names(out)) out[[target]] <- NA
  for (candidate in candidates) {
    if (!candidate %in% names(out)) next
    missing_target <- is.na(out[[target]])
    if (any(missing_target)) out[[target]][missing_target] <-
     ↪  out[[candidate]][missing_target]
  }
  out
}

normalize_ame_columns <- function(out) {
  # marginaleffects has used both dotted and snake_case names across versions
  # and model classes. Normalize here before selecting the public/audit schema
  # so uncertainty columns are not silently dropped downstream.
  aliases <- list(
    std.error = c("std_error", "std.err", "Std. Error"),
    statistic = c("z", "z.value", "z_value", "t", "t.value", "t_value"),
    p.value = c("p_value", "p", "Pr(>|z|)", "Pr(>|t|)"),
    s.value = c("s_value"),
    conf.low = c("conf_low", "conf.low", "2.5 %"),
    conf.high = c("conf_high", "conf.high", "97.5 %")
  )
  for (target in names(aliases)) {
    out <- copy_first_ame_column(out, target, aliases[[target]])
  }
  out
}
```

```r
ame_label_lookup <- function() {
  data.frame(
    term = c(
      "AGE", "SEX", "HH_SIZE",
      "RELIGION", "RELIGION", "RELIGION", "RELIGION", "RELIGION", "RELIGION", "RELIGION",
      "SOCIAL_GROUP", "SOCIAL_GROUP", "SOCIAL_GROUP",
      "SECTOR",
      "DIST_FROM_NEAREST_PRIMARY_CLASS", "DIST_FROM_NEAREST_PRIMARY_CLASS",
       ↪  "DIST_FROM_NEAREST_PRIMARY_CLASS", "DIST_FROM_NEAREST_PRIMARY_CLASS",
      "father_educ", "father_educ", "father_educ", "father_educ", "father_educ",
       ↪  "father_educ", "father_educ",
      "dmean_num_IS_EDU_FREE", "dmean_num_TUTION_FEE_WAIVED",
       ↪  "dmean_num_RECD_SCHOLARSHIP_STIPEND",
      "dmean_num_RECD_TXT_BOOKS", "dmean_num_RECD_STATIONERY",
       ↪  "dmean_num_MID_DAY_MEAL_ETC_RECD", "dmean_num_ENROLLMENT_COST"
    ),
    contrast = c(
      "dY/dX", "Female - Male", "dY/dX",
      "Muslim - Hindu", "Christian - Hindu", "Sikh - Hindu", "Jain - Hindu", "Buddhist -
       ↪  Hindu", "Zoroastrian - Hindu", "Other - Hindu",
      "Scheduled Tribe - Other", "Scheduled Caste - Other", "Other Backward Class -
       ↪  Other",
      "Urban - Rural",
      "1km <= d <2kms - d<1km", "2kms<= d <3kms - d<1km", "3kms <= d <5kms - d<1km",
       ↪  "d>=5kms - d<1km",
      "Literate, no school - Illiterate", "Literate, school < primary - Illiterate",
       ↪  "Primary - Illiterate", "Upper primary - Illiterate", "Secondary - Illiterate",
       ↪  "Higher secondary - Illiterate", "Postsecondary+ - Illiterate",
      "dY/dX", "dY/dX", "dY/dX", "dY/dX", "dY/dX", "dY/dX", "dY/dX"
    ),
    Term = c(
      "Age (years)", "Female (ref: Male)", "Household size",
      "Religion: Muslim (ref: Hindu)", "Religion: Christian", "Religion: Sikh",
       ↪  "Religion: Jain", "Religion: Buddhist", "Religion: Zoroastrian", "Religion:
       ↪  Other",
      "Social group: Scheduled Tribe (ref: Other)", "Social group: Scheduled Caste",
       ↪  "Social group: Other Backward Class",
      "Urban (ref: Rural)",
      "Distance 1-2km (ref: <1km)", "Distance 2-3km", "Distance 3-5km", "Distance > 5km",
      "Father's educ.: Literate, no school (ref: Illiterate)", "Father's educ.: Literate,
       ↪  school < primary", "Father's educ.: Primary", "Father's educ.: Upper primary",
       ↪  "Father's educ.: Secondary", "Father's educ.: Higher secondary", "Father's
       ↪  educ.: Postsecondary+",
      "Educ. free available (ref: No)", "Tuition waiver received", "Scholarship/Stipend
       ↪  received", "Textbook(s) received", "Stationery received", "Mid-day meal, etc.
       ↪  received", "Enrollment cost (Rs.)"
    ),
    stringsAsFactors = FALSE
  )
}

ame_label_order <- function() ame_label_lookup()$Term

ame_modelsummary_label <- function(x) {
  x <- as.character(x)
```

```
  x <- gsub("\u00d7", ":", x, fixed = TRUE)
  x <- gsub("\\s+:\\s+", ": ", x, perl = TRUE)
  x <- gsub("\\(ref:\\s*", "(ref: ", x, perl = TRUE)
  x
}

infer_ame_contrast <- function(out) {
  if ("contrast" %in% names(out)) return(as.character(out$contrast))
  contrast <- rep("dY/dX", nrow(out))
  if (!"term" %in% names(out)) return(contrast)
  if ("comparison" %in% names(out)) contrast <- as.character(out$comparison)
  contrast[is.na(contrast) | !nzchar(contrast)] <- "dY/dX"
  contrast
}

ame_factor_base_terms <- function() {
  unique(ame_label_lookup()$term[ame_label_lookup()$contrast != "dY/dX"])
}

normalize_encoded_factor_ame_terms <- function(out) {
  out <- as.data.frame(out, stringsAsFactors = FALSE)
  if (!"term" %in% names(out)) return(out)
  if (!"contrast" %in% names(out)) out$contrast <- infer_ame_contrast(out)

  lookup <- ame_label_lookup()
  factor_terms <- ame_factor_base_terms()
  factor_terms <- factor_terms[order(nchar(factor_terms), decreasing = TRUE)]

  for (i in seq_len(nrow(out))) {
    current_term <- as.character(out$term[[i]])
    current_contrast <- as.character(out$contrast[[i]])
    if (!nzchar(current_term) || (!is.na(current_contrast) && current_contrast !=
     ↪  "dY/dX")) next

    direct <- lookup$term == current_term & lookup$contrast == current_contrast
    if (any(direct)) next

    for (base_term in factor_terms) {
      if (!startsWith(current_term, base_term)) next
      level <- trimws(sub(paste0("^", base_term), "", current_term))
      if (!nzchar(level)) next
      candidates <- lookup[lookup$term == base_term, , drop = FALSE]
      contrast_hit <- candidates$contrast[startsWith(candidates$contrast, paste0(level, "
 ↪  - "))]
      if (length(contrast_hit)) {
        out$term[[i]] <- base_term
        out$contrast[[i]] <- contrast_hit[[1]]
        break
      }
    }
  }
  out
}

attach_ame_labels <- function(out) {
```

```
  out <- as.data.frame(out, stringsAsFactors = FALSE)
  if (!"term" %in% names(out) && "variable" %in% names(out)) out$term <- out$variable
  if (!"contrast" %in% names(out)) out$contrast <- infer_ame_contrast(out)
  out <- normalize_encoded_factor_ame_terms(out)
  lookup <- ame_label_lookup()
  labeled <- merge(out, lookup, by = c("term", "contrast"), all.x = TRUE, sort = FALSE)
  labeled$Term[is.na(labeled$Term)] <- labeled$term[is.na(labeled$Term)]
  labeled$Term <- factor(labeled$Term, levels = unique(c(ame_label_order(),
↪   labeled$Term)))
  labeled <- labeled[order(labeled$Term), , drop = FALSE]
  labeled$Term <- as.character(labeled$Term)
  labeled
}


modelsummary_marginaleffects_object <- function(out, native_ame) {
  if (!is_marginaleffects_object(native_ame)) return(NULL)
  out <- as.data.frame(out, stringsAsFactors = FALSE)
  required <- c("Term", "estimate", "std.error", "statistic", "p.value", "conf.low",
↪   "conf.high")
  if (!all(required %in% names(out))) return(native_ame)

  ms <- out[, required, drop = FALSE]
  ms$term <- ame_modelsummary_label(ms$Term)
  ms$contrast <- ""
  ms$Term <- NULL
  # Preserve the marginaleffects class and non-structural attributes so
  # modelsummary can use its native marginaleffects/tidy pathway.  We still
  # hand modelsummary the labeled, ordered rows that are validated for the
  # public table, because passing the raw object caused modelsummary to reorder
  # contrasts and display labels against the wrong estimates.
  native_attributes <- attributes(native_ame)
  structural_attributes <- c("names", "row.names", "class")
  for (nm in setdiff(names(native_attributes), structural_attributes)) {
    attr(ms, nm) <- native_attributes[[nm]]
  }
  class(ms) <- unique(c(class(native_ame), class(ms)))
  ms
}

#' format ame results
#'
format_ame_results <- function(ame_results) {
  native_ame <- ame_results
  out <- tibble::as_tibble(ame_results)
  if (!"term" %in% names(out) && "variable" %in% names(out)) out$term <- out$variable
  if (!"contrast" %in% names(out)) out$contrast <- infer_ame_contrast(as.data.frame(out))
  out <- normalize_ame_columns(out)
  if (!"method" %in% names(out)) out$method <- "autodiff"
  if (!"status" %in% names(out)) out$status <- "estimated"
  if (!"reason" %in% names(out)) out$reason <- NA_character_
  labeled_terms <- unique(ame_label_lookup()$term)
  use_labeled_schema <- any(out$term %in% labeled_terms) ||
    any(grepl("^dmean_num_", out$term %||% character(), perl = TRUE)) ||
    any((out$contrast %||% "dY/dX") != "dY/dX", na.rm = TRUE)
```

```r
  if (isTRUE(use_labeled_schema)) {
    out <- attach_ame_labels(out)
    required <- c(
      "Term", "term", "contrast", "estimate", "std.error", "statistic", "p.value",
       ↪  "s.value",
      "conf.low", "conf.high", "method", "status", "reason"
    )
  } else {
    required <- c(
      "term", "estimate", "std.error", "statistic", "p.value",
      "s.value", "conf.low", "conf.high", "method", "status", "reason"
    )
  }

  for (nm in setdiff(required, names(out))) out[[nm]] <- NA
  if ("p.value" %in% names(out) && "s.value" %in% names(out)) {
    missing_s <- is.na(out$s.value) & is.finite(out$p.value) & out$p.value > 0
    out$s.value[missing_s] <- -log2(out$p.value[missing_s])
  }
  out <- out[, required, drop = FALSE]
  if (is_marginaleffects_object(native_ame)) {
    attr(out, "marginaleffects_object") <- modelsummary_marginaleffects_object(out,
     ↪  native_ame)
  }
  out
}

#' save ame results
#'
save_ame_results <- function(ame_results, path =
 ↪  "outputs/tables/diagnostics/ame_results.csv") {
  readr::write_csv(tibble::as_tibble(ame_results), path); path
}
```

Cascading fuzzy district record linkage

Source: `R/districts/fuzzy_join_districts.R`

```r
# The functions below implement the current cascading fuzzy-match architecture.
# sample-end marker appears near the end of this file for coding-sample extraction.

#' fuzzy join sequence
#'
fuzzy_join_sequence <- function(df1, df2, dist1, state1, dist2, state2, methods,
 ↪  thresholds, mode = "full") {
  df1_id <- df1 |> dplyr::mutate(.id1 = dplyr::row_number())
  df2_id <- df2 |> dplyr::mutate(.id2 = dplyr::row_number())
  df1_curr <- df1_id; df2_curr <- df2_id; matched_all <- tibble::tibble()
  for (i in seq_along(methods)) {
    full_j <- fuzzyjoin::stringdist_join(df1_curr, df2_curr, by =
 ↪  stats::setNames(c(dist2, state2), c(dist1, state1)), mode = mode, method =
 ↪  methods[i], max_dist = thresholds[i], distance_col = "dist")
```

```
    matched_i <- full_j |> dplyr::filter(!is.na(.id1), !is.na(.id2)) |>
 ↪  dplyr::group_by(.id1) |> dplyr::slice_min(dist, n = 1, with_ties = FALSE) |>
 ↪  dplyr::ungroup() |> dplyr::group_by(.id2) |> dplyr::slice_min(dist, n = 1, with_ties
 ↪  = FALSE) |> dplyr::ungroup()
    matched_all <- dplyr::bind_rows(matched_all, matched_i)
    df1_curr <- df1_curr |> dplyr::filter(!(.id1 %in% matched_i$.id1)); df2_curr <-
 ↪  df2_curr |> dplyr::filter(!(.id2 %in% matched_i$.id2))
  }
  list(joined = matched_all, unmatched_df1 = df1_curr |> dplyr::select(-.id1),
   ↪  unmatched_df2 = df2_curr |> dplyr::select(-.id2))
}

#' unmatched rows
#'
#' @return A data frame of unmatched rows when the join map carries that attribute.
unmatched_rows <- function(join_map, ...) {
  attr(join_map, "unmatched_rows") %||% join_map[0, , drop = FALSE]
}

#' merge dfs into tracker
#'
#' Cascading district matcher. `df_names` is a named list (or a
#' data frame) whose elements carry source-specific district/state names.  For
#' each source we try every requested tracker year in order, keep one-to-one
#' canonical name matches, and then fuzzy-match remaining rows within state.
#'
#' @return A list with `joined_df`, `unmatched_df`, and `flagged_df`, matching
#' the current matcher object contract.
merge_dfs_into_tracker <- function(df_names, tracker = NULL, years_of_interest =
 ↪  c("2001", "2007", "2008", "2017", "2018", "2020"), flag = TRUE, ...) {
  if (is.null(tracker)) return(list(joined_df = safe_df(df_names), unmatched_df =
   ↪  data.frame(), flagged_df = data.frame()))
  tracker <- safe_df(tracker)
  if (!nrow(tracker)) return(list(joined_df = data.frame(), unmatched_df =
   ↪  safe_df(df_names), flagged_df = data.frame()))
  if (!".tracker_row" %in% names(tracker)) tracker$.tracker_row <- seq_len(nrow(tracker))
  xs <- if (inherits(df_names, "data.frame")) list(source = df_names) else
 ↪  as_input_list(df_names)

  joined_all <- list()
  unmatched_all <- list()
  flagged_all <- list()
  for (source_name in names(xs)) {
    source <- safe_df(xs[[source_name]])
    if (!nrow(source)) next
    source$.source_row <- seq_len(nrow(source))
    source$.source_name <- source_name
    source <- add_source_name_keys(source)
    if (!all(c(".source_state_key", ".source_district_key") %in% names(source))) {
      unmatched_all[[source_name]] <- source
      next
    }

    source$.matched <- FALSE
    for (yr in years_of_interest) {
```

```
      sfx <- substr(as.character(yr), 3, 4)
      state_col <- paste0("state_", sfx)
      district_col <- paste0("district_", sfx)
      if (!all(c(state_col, district_col) %in% names(tracker))) next
      candidate <- tracker[c(".tracker_row", state_col, district_col)]
      candidate$.tracker_state_key <- canon(candidate[[state_col]])
      candidate$.tracker_district_key <- canon(candidate[[district_col]])
      exact <- merge(
        source[!source$.matched, , drop = FALSE],
        candidate,
        by.x = c(".source_state_key", ".source_district_key"),
        by.y = c(".tracker_state_key", ".tracker_district_key"),
        all.x = FALSE,
        all.y = FALSE
      )
      if (!nrow(exact)) next
      exact <- exact[!duplicated(exact$.source_row) & !duplicated(exact$.tracker_row), ,
↪  drop = FALSE]
      if (!nrow(exact)) next
      exact$match_year <- as.character(yr)
      exact$match_status <- "exact_name"
      joined_all[[paste(source_name, yr, sep = "_")]] <- exact
      source$.matched[source$.source_row %in% exact$.source_row] <- TRUE
    }

    if (any(!source$.matched)) {
      fuzzy <- fuzzy_match_remaining_to_tracker(source[!source$.matched, , drop = FALSE],
↪  tracker, years_of_interest)
      if (nrow(fuzzy)) {
        joined_all[[paste0(source_name, "_fuzzy")]] <- fuzzy
        source$.matched[source$.source_row %in% fuzzy$.source_row] <- TRUE
        if (flag) flagged_all[[source_name]] <- fuzzy[fuzzy$possible_false_positive %in%
↪  TRUE, , drop = FALSE]
      }
    }
    unmatched_all[[source_name]] <- source[!source$.matched, , drop = FALSE]
  }
  list(
    joined_df = safe_bind_rows(joined_all),
    unmatched_df = safe_bind_rows(unmatched_all),
    flagged_df = safe_bind_rows(flagged_all)
  )
}

add_source_name_keys <- function(source) {
  state <- first_col(source, c("state", "state_01", "state_07", "state_08", "state_17",
↪  "state_18", "state_20", "state_0708", "state_1718"))
  district <- first_col(source, c("district", "district_01", "district_07",
↪  "district_08", "district_17", "district_18", "district_20", "district_0708",
↪  "district_1718", "district_name"))
  if (!is.null(state)) source$.source_state_key <- canon(source[[state]])
  if (!is.null(district)) source$.source_district_key <- canon(source[[district]])
  source
}
```

```r
fuzzy_match_remaining_to_tracker <- function(source, tracker, years_of_interest) {
  out <- list()
  for (yr in years_of_interest) {
    sfx <- substr(as.character(yr), 3, 4)
    state_col <- paste0("state_", sfx)
    district_col <- paste0("district_", sfx)
    if (!all(c(state_col, district_col) %in% names(tracker))) next
    candidate <- tracker[c(".tracker_row", state_col, district_col)]
    candidate$.tracker_state_key <- canon(candidate[[state_col]])
    candidate$.tracker_district_key <- canon(candidate[[district_col]])
    for (i in seq_len(nrow(source))) {
      pool <- candidate[candidate$.tracker_state_key == source$.source_state_key[[i]], ,
 ↪  drop = FALSE]
      if (!nrow(pool)) next
      dist <- utils::adist(source$.source_district_key[[i]], pool$.tracker_district_key,
 ↪  ignore.case = TRUE)[1, ]
      best <- which.min(dist)
      if (!length(best) || !is.finite(dist[[best]]) || dist[[best]] > 2) next
      row <- cbind(source[i, , drop = FALSE], pool[best, , drop = FALSE])
      row$match_year <- as.character(yr)
      row$match_status <- "fuzzy_name"
      row$dist <- dist[[best]]
      row$possible_false_positive <- dist[[best]] > 0
      out[[length(out) + 1L]] <- row
    }
  }
  x <- safe_bind_rows(out)
  if (nrow(x)) x <- x[!duplicated(x$.source_row) & !duplicated(x$.tracker_row), , drop =
   ↪  FALSE]
  x
}

#' join year to tracker
#'
join_year_to_tracker <- function(source, tracker, years_of_interest, ...) {
  merge_dfs_into_tracker(source, tracker = tracker, years_of_interest =
   ↪  years_of_interest, ...)
}

#' join all district sources
#'
join_all_district_sources <- function(df_names, tracker, years_of_interest, ...) {
  merge_dfs_into_tracker(df_names, tracker = tracker, years_of_interest =
   ↪  years_of_interest, ...)
}

#' flag many to many matches
#'
flag_many_to_many_matches <- function(join_map, source_col = NULL, tracker_col = NULL) {
  join_map <- safe_df(join_map)
  if (!nrow(join_map)) {
    join_map$many_to_many <- logical()
    join_map$many_to_many_type <- character()
    return(join_map)
  }
```

```
  source_col <- source_col %||% first_col(join_map, c(".source_row", "source_row",
 ↪  "source_key", "source_id"))
  tracker_col <- tracker_col %||% first_col(join_map, c(".tracker_row", "tracker_row",
 ↪  "tracker_key", "district_panel_id"))
  if (is.null(source_col) || is.null(tracker_col)) {
    join_map$many_to_many <- FALSE
    join_map$many_to_many_type <- "not_evaluable_missing_keys"
    return(join_map)
  }
  source_key <- canon(join_map[[source_col]])
  tracker_key <- canon(join_map[[tracker_col]])
  source_n <- ave(rep(1L, nrow(join_map)), source_key, FUN = length)
  tracker_n <- ave(rep(1L, nrow(join_map)), tracker_key, FUN = length)
  source_dup <- source_n > 1L & !is.na(source_key) & nzchar(source_key)
  tracker_dup <- tracker_n > 1L & !is.na(tracker_key) & nzchar(tracker_key)
  join_map$n_source_matches <- as.integer(source_n)
  join_map$n_tracker_matches <- as.integer(tracker_n)
  join_map$many_to_many <- source_dup & tracker_dup
  join_map$many_to_many_type <- ifelse(
    source_dup & tracker_dup, "many_to_many",
    ifelse(source_dup, "one_source_to_many_tracker",
      ifelse(tracker_dup, "many_source_to_one_tracker", "one_to_one"))
  )
  join_map
}

#' score candidate matches
#'
score_candidate_matches <- function(candidates) {
  if (!"dist" %in% names(candidates)) candidates$dist <- 0
  candidates$score <- -num(candidates$dist)
  candidates
}

#' fuzzy join districts
#'
fuzzy_join_districts <- function(district_tracker, district_keys_2001,
 ↪  district_keys_2007, district_keys_2017, district_keys_2020, cfg) {
  tracker <- safe_df(district_tracker)
  if (nrow(tracker) && all(c("state_01", "district_01", "state_07", "district_07",
   ↪  "state_17", "district_17", "state_20", "district_20") %in% names(tracker))) {
    tracker$.tracker_row <- seq_len(nrow(tracker))
    tracker$source <- "harmonization_crosswalk"
    tracker$match_status <- "harmonization_crosswalk_row"
    tracker$possible_false_positive <- FALSE
    tracker$many_to_many <- FALSE
    attr(tracker, "unmatched_rows") <- tracker[0, , drop = FALSE]
    attr(tracker, "possible_false_positives") <- tracker[0, , drop = FALSE]
    attr(tracker, "many_to_many_cases") <- tracker[0, , drop = FALSE]
    return(tracker)
  }

  out <- safe_bind_rows(Map(function(keys, source) {
    if (!nrow(keys)) return(data.frame())
    keys$source <- source
```

```
    keys$match_status <- "source_key_unmatched"
    keys$possible_false_positive <- FALSE
    keys$many_to_many <- FALSE
    keys
  }, list(district_keys_2001, district_keys_2007, district_keys_2017,
↪  district_keys_2020), c("2001", "2007", "2017", "2020")))
  attr(out, "unmatched_rows") <- out
  attr(out, "possible_false_positives") <- out[out$possible_false_positive, , drop =
↪  FALSE]
  attr(out, "many_to_many_cases") <- out[out$many_to_many, , drop = FALSE]
  out
}
```

Contiguity-based spatial weights

Source: `R/diagnostics/diagnose_spatial_weights.R`

```
#' build spatial weights
#'
#' Build the rook-contiguity spatial weights used by the legacy Rmd.
#'
#' The legacy chunk first removed rows with missing analysis values, then called
#' `poly2nb(queen = FALSE)`, `nb2mat(style = "B")`, and `nb2listw(style = "W")`.
#' The target stores all three objects so downstream diagnostics can reproduce
#' both the binary adjacency matrix and the row-standardized listw object.
#'
#' @return A list containing neighbor, matrix, and listw objects, or an explicit
#' inactive status when geometry is unavailable.
build_spatial_weights <- function(district_panel, cfg) {
  if (!inherits(district_panel, "sf")) {
    return(list(status = "out_of_active_pipeline", reason = "Requires sf geometry."))
  }
  need_pkg("spdep", "spatial weights")
  need_pkg("sf", "spatial weights")

  row_index <- spatial_weight_final_panel_rows(district_panel)
  coverage <- length(row_index) / max(nrow(district_panel), 1L)
  if (!is.finite(coverage) || coverage < 0.75) {
    return(list(
      status = "out_of_active_pipeline",
      reason = paste0(
        "Requires validated non-empty geometry for at least 75% of district-panel rows;
↪  current coverage is ",
        round(100 * coverage, 1), "%.")
    ))
  }

  weights <- build_spatial_weights_for_rows(district_panel, row_index, queen = FALSE)
  weights$coverage <- coverage
  weights$panel_scope <- "current_final_matched_panel_non_empty_geometry"
  weights
}

#' rows used by final-panel spatial diagnostics
#'
#' Spatial diagnostics operate on the active `district_panel` target, not on the
```

```r
#' legacy exploratory geometry object.  The final-panel scope is therefore all
#' rows in the current matched panel with non-empty geometry.
spatial_weight_final_panel_rows <- function(district_panel) {
  if (!inherits(district_panel, "sf")) return(integer())
  geom <- sf::st_geometry(district_panel)
  which(!sf::st_is_empty(geom))
}

#' build spatial weights for selected district-panel rows
#'
#' @return A list with nb, binary matrix, and row-standardized listw objects.
build_spatial_weights_for_rows <- function(district_panel, rows, queen = FALSE) {
  if (!inherits(district_panel, "sf")) {
    return(list(status = "out_of_active_pipeline", reason = "Requires sf geometry."))
  }
  need_pkg("spdep", "spatial weights")
  need_pkg("sf", "spatial weights")

  rows <- as.integer(rows)
  rows <- rows[is.finite(rows) & rows >= 1L & rows <= nrow(district_panel)]
  if (!length(rows)) {
    return(list(status = "out_of_active_pipeline", reason = "No rows with usable
    ↪  geometry."))
  }
  geom <- sf::st_geometry(district_panel)
  rows <- rows[!sf::st_is_empty(geom[rows])]
  if (!length(rows)) {
    return(list(status = "out_of_active_pipeline", reason = "Selected rows have empty
    ↪  geometry."))
  }

  spatial_warnings <- character()
  capture_expected_spatial_warning <- function(expr) {
    withCallingHandlers(
      expr,
      warning = function(w) {
        msg <- conditionMessage(w)
        expected <- grepl("no neighbours|sub-graphs", msg, ignore.case = TRUE)
        if (expected) {
          spatial_warnings <<- unique(c(spatial_warnings, msg))
          invokeRestart("muffleWarning")
        }
      }
    )
  }

  panel <- district_panel[rows, , drop = FALSE]
  nb <- capture_expected_spatial_warning(spdep::poly2nb(panel, queen = queen))
  W <- capture_expected_spatial_warning(spdep::nb2mat(nb, style = "B", zero.policy =
↪  TRUE))
  listw <- capture_expected_spatial_warning(spdep::nb2listw(nb, style = "W", zero.policy
↪  = TRUE))

  out <- list(
    status = "constructed",
```

```
    contiguity = if (isTRUE(queen)) "queen" else "rook",
    style = "W",
    matrix_style = "B",
    zero_policy = TRUE,
    row_index = rows,
    nb = nb,
    W = W,
    listw = listw,
    neighbor_counts = lengths(nb),
    n = length(rows),
    n_islands = sum(lengths(nb) == 0L),
    mean_neighbors = mean(lengths(nb)),
    warnings = spatial_warnings
  )
  class(out) <- c("emi_spatial_weights", class(out))
  out
}

#' diagnose spatial weights
#'
diagnose_spatial_weights <- function(district_panel, spatial_weights, cfg) {
  if (!inherits(district_panel, "sf")) {
    return(data.frame(
      diagnostic = "spatial_weights",
      status = "out_of_active_pipeline",
      reason = "Requires sf geometry.",
      stringsAsFactors = FALSE
    ))
  }

  comparison <- compare_rook_queen_contiguity(district_panel)
  base <- data.frame(
    diagnostic = "spatial_weights",
    status = spatial_weights$status %||% "constructed",
    contiguity = spatial_weights$contiguity %||% NA_character_,
    style = spatial_weights$style %||% NA_character_,
    matrix_style = spatial_weights$matrix_style %||% NA_character_,
    n = spatial_weights$n %||% NA_real_,
    n_islands = spatial_weights$n_islands %||% NA_real_,
    mean_neighbors = spatial_weights$mean_neighbors %||% NA_real_,
    panel_scope = spatial_weights$panel_scope %||%
      ↪ "current_final_matched_panel_non_empty_geometry",
    warnings = paste(spatial_weights$warnings %||% attr(spatial_weights,
      ↪ "spatial_warnings") %||% character(), collapse = "; "),
    stringsAsFactors = FALSE
  )
  attr(base, "rook_queen_comparison") <- add_spatial_weight_reference(comparison)
  attr(base, "legacy_reference") <- spatial_weight_reference()
  base
}

#' compare rook and queen contiguity
#'
#' The legacy Rmd commented out an sfExtras rook-vs-queen check and recorded
#' nearly identical mean neighbor counts (rook 4.780165, queen 4.783471) plus
```

```r
#' similar run times.  To avoid an extra sfExtras dependency, this project uses
#' the same `spdep::poly2nb()` implementation used by the final rook weights and
#' records elapsed time and average neighbor counts for both choices.
compare_rook_queen_contiguity <- function(district_panel) {
  if (!inherits(district_panel, "sf")) return(tibble::tibble())
  need_pkg("spdep", "rook/queen contiguity comparison")
  need_pkg("sf", "rook/queen contiguity comparison")
  panel <- district_panel[spatial_weight_final_panel_rows(district_panel), , drop =
 ↪  FALSE]
  if (!nrow(panel)) return(tibble::tibble())

  one <- function(queen) {
    warnings <- character()
    elapsed <- system.time({
      nb <- withCallingHandlers(
        spdep::poly2nb(panel, queen = queen),
        warning = function(w) {
          msg <- conditionMessage(w)
          if (grepl("no neighbours|sub-graphs", msg, ignore.case = TRUE)) {
            warnings <<- unique(c(warnings, msg))
            invokeRestart("muffleWarning")
          }
        }
      )
    })[["elapsed"]]
    tibble::tibble(
      contiguity = if (isTRUE(queen)) "queen" else "rook",
      n = length(nb),
      mean_neighbors = mean(lengths(nb)),
      n_islands = sum(lengths(nb) == 0L),
      panel_scope = "current_final_matched_panel_non_empty_geometry",
      elapsed_seconds = unname(elapsed),
      warnings = paste(warnings, collapse = "; ")
    )
  }

  safe_bind_rows(list(one(FALSE), one(TRUE)))
}


spatial_weight_reference <- function() {
  data.frame(
    contiguity = c("rook", "queen"),
    legacy_method = c(
      "sfExtras::st_rook() timing comment; final weights use spdep::poly2nb(queen =
       ↪  FALSE)",
      "sfExtras::st_queen() timing comment; benchmark uses spdep::poly2nb(queen = TRUE)"
    ),
    legacy_mean_neighbors = c(4.780165, 4.783471),
    legacy_elapsed_note = c("legacy comment recorded similar run time to queen", "legacy
     ↪  comment recorded similar run time to rook"),
    interpretation = "Current means are intentionally computed on the active final
     ↪  matched panel with non-empty geometry; they may differ from legacy exploratory
     ↪  sfExtras objects, but they now match the panel used for current spatial
     ↪  diagnostics.",
```

```
    stringsAsFactors = FALSE
  )
}

add_spatial_weight_reference <- function(comparison) {
  comparison <- as.data.frame(comparison, stringsAsFactors = FALSE)
  if (!nrow(comparison)) return(comparison)
  ref <- spatial_weight_reference()[c("contiguity", "legacy_mean_neighbors")]
  out <- merge(comparison, ref, by = "contiguity", all.x = TRUE, sort = FALSE)
  out$mean_neighbor_delta_from_legacy <- out$mean_neighbors - out$legacy_mean_neighbors
  out$pct_delta_from_legacy <- 100 * out$mean_neighbor_delta_from_legacy /
 ↪  out$legacy_mean_neighbors
  out
}

#' summarize islands
#'
summarize_islands <- function(spatial_weights) {
  if (!inherits(spatial_weights, "emi_spatial_weights")) return(tibble::tibble())
  islands <- which(spatial_weights$neighbor_counts == 0L)
  tibble::tibble(row_index = spatial_weights$row_index[islands], n_neighbors = 0L)
}

#' summarize neighbor counts
#'
summarize_neighbor_counts <- function(spatial_weights) {
  if (!inherits(spatial_weights, "emi_spatial_weights")) return(tibble::tibble())
  tibble::tibble(
    row_index = spatial_weights$row_index,
    n_neighbors = spatial_weights$neighbor_counts
  )
}

#' save spatial weight diagnostics
#'
save_spatial_weight_diagnostics <- function(diagnostics, dir =
 ↪  "outputs/diagnostics/extended/spatial") {
  dir.create(dir, recursive = TRUE, showWarnings = FALSE)
  paths <- c(
    summary = write_diagnostic_csv(as.data.frame(diagnostics), file.path(dir,
     ↪  "spatial_weights_summary.csv")),
    rook_queen_comparison = write_diagnostic_csv(attr(diagnostics,
     ↪  "rook_queen_comparison") %||% data.frame(), file.path(dir,
     ↪  "rook_queen_contiguity_comparison.csv")),
    legacy_reference = write_diagnostic_csv(attr(diagnostics, "legacy_reference") %||%
     ↪  data.frame(), file.path(dir, "spatial_weights_legacy_reference.csv"))
  )
  output_manifest(paths)
}
```

Reusable IV specification and estimation

Source: `R/iv/build_iv_formulas.R`

```
#' make iv formula
```

```r
#'
make_iv_formula <- function(dep, endog, instruments, controls = NULL, fixed_effects =
 ↪  NULL) {
  stats::as.formula(paste(
    dep,
    "~",
    paste(c(endog, controls, fixed_effects), collapse = " + "),
    "|",
    paste(c(instruments, controls, fixed_effects), collapse = " + ")
  ))
}

baseline_iv_controls <- function() {
  c(
    "consumption_0708", "gini_cons_0708",
    "pct_urban", "avg_hh_size", "dependency_ratio",
    "pct_fem_head",
    "pct_hindu", "pct_muslim",
    "pct_st", "pct_sc", "pct_obc",
    "pct_small_land", "pct_medium_land", "pct_large_land",
    "pct_head_lit_to_primary", "pct_head_secondary_plus"
  )
}

#' build iv formulas
#'
build_iv_formulas <- function(cfg) {
  controls <- baseline_iv_controls()
  list(
    consumption = make_iv_formula(
      "consumption_pct_change",
      "EMIE",
      "wavg_ling_degrees",
      controls
    ),
    gini = make_iv_formula(
      "gini_change",
      "EMIE",
      "wavg_ling_degrees",
      controls
    )
  )
}

#' build baseline 2sls formula
#'
build_baseline_2sls_formula <- function(...) make_iv_formula(...)
```

Multicollinearity and spatial autocorrelation diagnostics

Source: `R/diagnostics/diagnose_multicollinearity.R`

```r
#' diagnose multicollinearity
#'
#' @return A data frame with design-matrix condition diagnostics.
diagnose_multicollinearity <- function(district_panel, iv_models, cfg) {
```

```
  model <- if (is.list(iv_models) && length(iv_models)) iv_models[[1]] else iv_models
  out <- tryCatch({
    X <- stats::model.matrix(model)
    data.frame(
      test = "kappa",
      n = nrow(X),
      rank = qr(X)$rank,
      columns = ncol(X),
      kappa = kappa(X, exact = TRUE),
      status = "estimated",
      reason = NA_character_,
      stringsAsFactors = FALSE
    )
  }, error = function(e) {
    data.frame(
      test = "kappa",
      n = NA_integer_,
      rank = NA_integer_,
      columns = NA_integer_,
      kappa = NA_real_,
      status = "out_of_active_pipeline",
      reason = conditionMessage(e),
      stringsAsFactors = FALSE
    )
  })
  out
}

#' compute design matrix rank
#'
compute_design_matrix_rank <- function(formula, data) {
  X <- stats::model.matrix(formula, data = data); tibble::tibble(rank = qr(X)$rank, ncol
 ↪  = ncol(X))
}

#' compute kappa
#'
compute_kappa <- function(formula, data) {
  X <- stats::model.matrix(formula, data = data); kappa(X, exact = TRUE)
}

#' compute vif if applicable
#'
compute_vif_if_applicable <- function(model) {
  if (requireNamespace("car", quietly = TRUE)) car::vif(model) else NULL
}

#' save multicollinearity diagnostics
#'
save_multicollinearity_diagnostics <- function(diagnostics) {
  diagnostics
}
```